

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 3 :

Réversivité

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Définition (et pourquoi)
2. Exemples
3. Cas de base et cas récursive
4. Autres exemples

Définition

Récurtivité: c'est quoi, et pourquoi

- Fonction qui fait appel à elle-même
- C'est une autre approche, souvent plus élégante, pour écrire des algorithmes (surtout pour remplacer l'approche itérative)
- C'est une approche qui se base sur le principe de "diviser et conquérir"
 - c.à.d., diviser un problème complexe en sous problèmes faciles, résoudre les sous-problèmes, et utiliser ces solutions pour résoudre le problème original
- Aussi, parfois, on peut rencontrer des problèmes qui sont intrinsèquement récursifs
 - c.à.d., ils n'acceptent que la solution récursive (ou bien la solution itérative est très complexe)
 - Nous allons voir un exemple de tels problèmes vers la fin de ce chapitre



Exemple 1:

multiplication

Multiplication:

Approche itérative

- $3 * 4 = 3 + 3 + 3 + 3$

```
donc : resultat = 0
      for i in range(4):
          result = resultat + a
      return resultat
```

```
## itérative:
def multiply_iterative(a, b):
    result = 0
    for i in range(b):
        result += a
    return result

result = multiply_iterative(4, 3)
print(result)
```

12

Multiplication:

Approche récursive

- $3 * 4 = 3 + 3 + 3 + 3$
 $= 3 + [3 * (4-1)] = 3 + [3 * 3]$
- $[3 * 3] = 3 + 3 + 3$
 $= 3 + [3 * 2]$
- $[3 * 2] = 3 + 3$
 $= 3 + [3 * 1]$
- $[3 * 1] = 3$

```
def mult(a, b):  
    # cas de base  
    if b == 1:  
        return a  
    # cas récursive  
    else:  
        return a + mult(a, b - 1)  
  
result = mult(4, 3)  
print(result)
```

12

Cas de base : Condition pour laquelle la fonction doit arrêter d'appeler elle-même.
Très important de la définir.

Note :

- Si on définit pas le cas de base, la fonction va donc appeler elle-même **infiniment** !!!!

► # Exemple:

```
def foo(x):  
    foo(x)  
  
foo(3)
```

```
Traceback (most recent call last):  
  File "<pyshell#8>", line 1, in <module>  
    foo(3)  
  File "<pyshell#7>", line 2, in foo  
    foo(x)  
  File "<pyshell#7>", line 2, in foo  
    foo(x)  
  File "<pyshell#7>", line 2, in foo  
    foo(x)  
  [Previous line repeated 1022 more times]  
RecursionError: maximum recursion depth exceeded
```

```
>>> |
```


Maintenant, Comment marche le programme de multiplication récursive

```
def mult(a, b):  
    # cas de base  
    if b == 1:  
        return a  
    # cas récursive  
    else:  
        return a + mult(a, b - 1)  
  
result = mult(4, 3)  
print(result)
```

12

a = 3, b = 4

mult(3, 4)

a + mult(a, b - 1) #b=3

a + mult(a, b - 1) #b=2

a + mult(a, b - 1) #b=1

return a

3 + quelque chose qu'on va calculer, qui égale à :

3 + quelque chose qu'on va calculer, qui égale à :

3 + quelque chose qu'on va calculer, qui égale à :

3

et donc on remplace le résultat de chaque terme du dernier vers le premier, on aura : 12

Exemple 2:

Factoriel

Factoriel:

Approche itérative

$5! = 5 * 4 * 3 * 2 * 1$ ## ou l'inverse : $= 1 * 2 * 3 * 4 * 5$

Donc, on multiplie de 1 jusqu'au nombre pour lequel on veut faire le factoriel :

```
n = 1
for i in range(1, 6):
    n = n * i

return n
```

```
def factorial_iterative(n):
    if n < 0:
        return None #pas de factoriel pour les nombres négatives
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

```
result = factorial_iterative(5)
print(result)
```

120

Essayer de la faire de manière récursive,

Indication: $5! = 5 * 4 * 3 * 2 * 1$
 $= 5 * 4 !$

Factoriel:

Approche récursive : en théorie

$$5 ! = 5 * 4 !$$

$$\text{avec } 4 ! = 4 * 3 !$$

$$\text{avec } 3 ! = 3 * 2 !$$

$$\text{avec } 2 ! = 2 * 1 !$$

$$\text{avec } 1 ! = 1$$

Maintenant on remonte en haut et on remplace:

$$2 ! = 2 * 1 \quad (\text{car } 1!=1) \quad \text{donc c'est} = 2$$

$$3 ! = 3 * 2 ! = 3 * 2 = 6$$

$$4 ! = 4 * 3 ! = 4 * 6 = 24$$

$$5 ! = 5 * 4 ! = 5 * 24 = 120$$

Factoriel:

Approche récursive : en code

```
def factorial_recursive(n):  
    # Cas de base : fact(0) = fact(1) = 1  
    if n == 0 or n == 1:  
        return 1  
    # Cas récursive:  $n! = n * (n-1)!$   
    else:  
        return n * factorial_recursive(n-1)  
  
result = factorial_recursive(5)  
print(result)
```

120

Exemple 3:

Fibonacci

Fibonacci:

Approche itérative

```
def fibonacci_iterative(n):  
    if n<=1:  
        return n  
  
    a, b = 0, 1  
    for i in range(2, n+1):  
        res = a + b  
        a = b  
        b = res  
  
    return res  
  
s = time.time()  
#*****  
res = fibonacci_iterative(6)  
print(res)  
#*****  
f = time.time()  
print(f - s, 'sec')
```

```
8  
0.007388114929199219 sec
```

Séquence de Fibonacci

0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 +

$pos_i = (pos_{i-1}) + (pos_{i-2})$

fibonacci_iterative(6) invoque range(2, 7)
donc 5 itérations pour incrémenter a et b et res:

Voici les 5 itérations:

```
0 + 1  
1 + 1  
1 + 2  
2 + 3  
3 + 5
```


Essayer de la faire de manière récursive,

Indication :

fib = 1 + 1 + 2 + 3 + 5 + 8 + 13 + ...

Fib(n) = fib(n-1) + fib(n-2)

Fibonacci:

Approche récursive naïve

```
def fib_rec(n):  
    if n <= 1:  
        return n  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)  
  
n = 35  
  
s = time.time()  
#*****  
res = fib_rec(n)  
print(res)  
#*****  
f = time.time()  
print(f - s, 'sec')
```

9227465

8.79795503616333 sec

Fibonacci:

Approche récursive **naïve**

```
def fib_rec(n):  
    if n <= 1:  
        return n  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

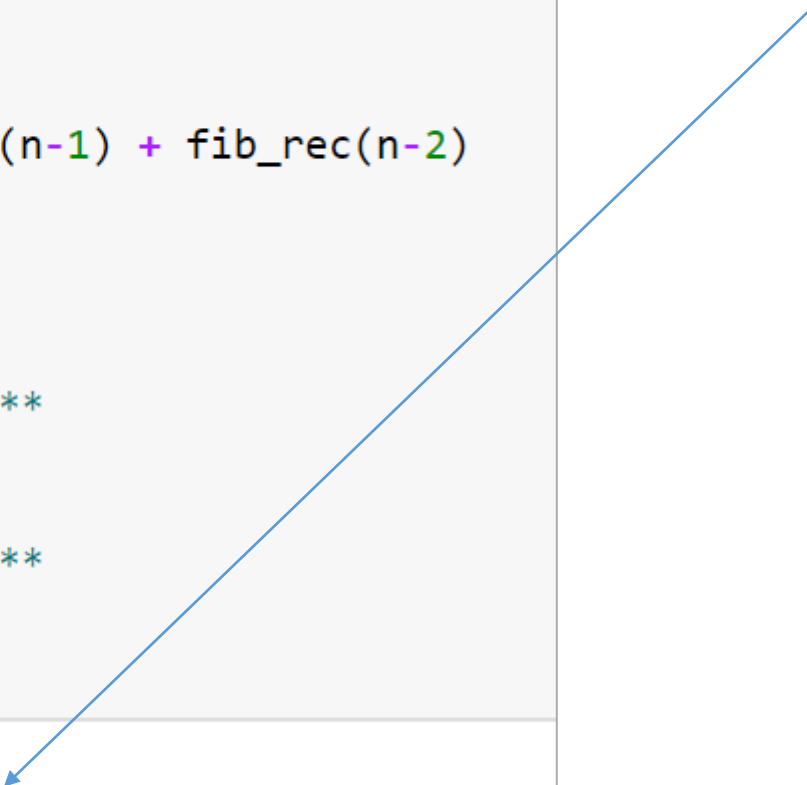
```
n = 35
```

```
s = time.time()  
*****  
res = fib_rec(n)  
print(res)  
*****  
f = time.time()  
print(f - s, 'sec')
```

```
9227465
```

```
8.79795503616333 sec
```

Prend beaucoup de temps à cause
des appels imbriqués répétitifs



Fibonacci:

Approche réursive naïve

Cette approche naïve génère fait des appels redondants inutiles

- Pourquoi ça prend beaucoup de temps ??

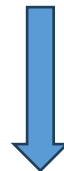
```
idx = 1  2  3  4  5  6  7  ...  
res = 1 + 1 + 2 + 3 + 5 + 8 + 13 ...
```

- Si on veut calculer le 5ème terme, on doit faire:

```
fibonacci_recursive(5) = fibonacci_recursive(4) + fibonacci_recursive(3)  
                        fibonacci_recursive(3) + fibonacci_recursive(2) + fibonacci_recursive(2) + fibonacci_recursive(1)
```

- etc..., chaque terme appelle la fonction deux fois, et ces deux appels vont chacun appeler deux fois, etc.

Solution ?



Fibonacci:

Approche récursive + mémoïsation

Éliminer les calculs redondants en les sauvegardant dans un dictionnaire

```
memo = {}

def fibonacci_recursive_memo(n):
    ## si le n à calculer est dans notre dictionnaire --> retourne le
    if n in memo:
        return memo[n]

    ## sinon, calcule le, sauvegarde le dans memo, et retourne le
    if n <= 1:
        return n ## résultat pour les indices 0 et 1

    else:
        memo[n] = fibonacci_recursive_memo(n-1) + fibonacci_recursive_memo(n-2)

    return memo[n]

n = 35

s = time.time()
*****
res = fibonacci_recursive_memo(n)
print(res)
*****
f = time.time()
print(f - s, 'sec')
```

```
9227465
0.005891561508178711 sec
```